

UNIVERSIDADE TUIUTI DO PARANÁ

**CAIO FEDERICO ESQUIVEL LOVERA ARZE
LUCAS TOTEROL RODRIGUES**

**RELATÓRIO DE ESTUDO DIRIGIDO:
PLAYER DE MÚSICA EM C**

**CURITIBA
2026**

**CAIO FEDERICO ESQUIVEL LOVERA ARZE
LUCAS TOTEROL RODRIGUES**

**RELATÓRIO DE ESTUDO DIRIGIDO:
PLAYER DE MÚSICA EM C**

Estudo dirigido apresentado ao curso de Análise e Desenvolvimento de Sistemas, da Universidade Tuiuti do Paraná, como requisito avaliativo do 2º bimestre da disciplina de Estrutura de Dados.

Orientador: Prof. Luiz Adolpho Baroni

**CURITIBA
2026**

RESUMO

Este trabalho descreve o desenvolvimento de um player de música interativo escrito em linguagem C. A ideia foi colocar em prática os conceitos de estruturas de dados dinâmicas vistos no bimestre, ou seja, lista duplamente encadeada, pilha e fila, todas montadas com alocação dinâmica de memória. A biblioteca de músicas é guardada em uma lista duplamente encadeada, o histórico de reprodução é uma pilha encadeada e a fila de reprodução é uma fila encadeada. As playlists também usam lista duplamente encadeada, sendo que cada playlist guarda, por dentro, uma lista de invólucros que apontam para as músicas da biblioteca sem copiá-las. Para que o player realmente tocasse os arquivos, integramos a biblioteca miniaudio, e a leitura de teclado sem bloqueio permite controlar a reprodução ao vivo. O código ficou separado em módulos, com um diretório por responsabilidade. Ao longo do caminho aparecerem algumas dificuldades técnicas que valem registro, como impedir a remoção de músicas que ainda estão em uso, evitar dependência circular entre os módulos, fazer a integração do áudio em tempo real e implementar a entrada de teclado não bloqueante funcionando nos dois sistemas operacionais. No fim, o sistema ficou funcional, com menu interativo, reprodução de fato dos arquivos e navegação completa entre músicas, playlists e histórico.

Palavras-chave: Estruturas de dados; Lista duplamente encadeada; Pilha; Fila; Linguagem C; Player de música.

LISTA DE SIGLAS E ACRÔNIMOS

API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
CLI	<i>Command Line Interface</i> (Interface de Linha de Comando)
ED	Estudo Dirigido
FIFO	<i>First In, First Out</i> (Primeiro a Entrar, Primeiro a Sair)
IA	Inteligência Artificial
LIFO	<i>Last In, First Out</i> (Último a Entrar, Primeiro a Sair)

SUMÁRIO

1	INTRODUÇÃO	6
1.1	OBJETIVOS	6
1.2	DESCRIÇÃO DO PROBLEMA	6
2	FUNDAMENTAÇÃO TEÓRICA	8
2.1	ESTRUTURA DE DADOS: LISTA DUPLAMENTE ENCADEADA	8
2.2	ESTRUTURA DE DADOS: PILHA ENCADEADA	8
2.3	ESTRUTURA DE DADOS: FILA ENCADEADA	8
2.4	GERENCIAMENTO DE MEMÓRIA E POLÍTICA DE REMOÇÃO	9
2.5	A ESCOLHA DAS ESTRUTURAS	9
3	PLANEJAMENTO INICIAL	10
3.1	INTERPRETAÇÃO DO PROBLEMA	10
3.2	ESTRUTURAS QUE SERÃO UTILIZADAS	10
4	IMPLEMENTAÇÃO	12
4.1	INICIALIZAÇÃO DO REPOSITÓRIO DO GITHUB	12
4.2	ESTRUTURA DE ARQUIVOS E MAKEFILE	12
4.3	IMPLEMENTAÇÃO DA BIBLIOTECA (LISTA DUPLAMENTE ENCADEADA)	13
4.4	IMPLEMENTAÇÃO DA PILHA ENCADEADA	13
4.5	IMPLEMENTAÇÃO DA FILA ENCADEADA	13
4.6	IMPLEMENTAÇÃO DAS PLAYLISTS	13
4.7	INTEGRAÇÃO DE ÁUDIO REAL	13
4.8	CONTROLE DE TECLADO NÃO BLOQUEANTE	14
4.9	LOOP DE REPRODUÇÃO AO VIVO	14
4.10	TESTES E VALIDAÇÕES	14
4.11	DIFICULDADES E APRENDIZADOS	15
4.11.1	Gerenciamento de Propriedade de Memória	15
4.11.2	Dependência Circular entre Módulos	15
4.11.3	Wrapper de EntradaPlaylist	15
4.11.4	Integração da Biblioteca miniaudio	15
4.11.5	Teclado Não Bloqueante Multiplataforma	16
4.11.6	Restauração Segura do Terminal	16
4.11.7	Pausa Portável sem usleep	16
4.11.8	Tratamento de Buffer de Entrada nos Menus	16
4.12	DECISÕES FEITAS DURANTE O DESENVOLVIMENTO	16
4.13	USO DE IA NO PROJETO	17
5	TECNOLOGIAS UTILIZADAS	18
6	DISTRIBUIÇÃO DE TAREFAS	19
7	CONSIDERAÇÕES FINAIS	21

REFERÊNCIAS	22
--------------------	-----------

1 INTRODUÇÃO

Neste estudo dirigido desenvolvemos um player de música interativo em linguagem C, com a intenção de aplicar na prática os conceitos de estruturas de dados dinâmicas trabalhados no segundo bimestre. A diferença em relação ao estudo anterior, que usava vetores de tamanho fixo, é que aqui passamos a trabalhar com alocação dinâmica de memória, montando listas encadeadas, pilhas e filas a partir de ponteiros.

O sistema deixa o usuário cadastrar músicas em uma biblioteca, organizá-las em playlists, colocar músicas na fila para tocar em sequência e voltar no histórico de reprodução. A reprodução de áudio acontece de verdade, usando a biblioteca miniaudio, e o controle durante a execução é feito direto pelo teclado, sem precisar apertar ENTER a cada comando.

O objetivo central foi mostrar como estruturas de dados dinâmicas podem ser combinadas para resolver um problema concreto. Com isso também conseguimos evidenciar as vantagens do encadeamento em memória, como não ter um limite fixo de capacidade e poder reaproveitar os nodos conforme a necessidade, algo que as implementações estáticas com vetores não oferecem.

1.1 OBJETIVOS

O objetivo geral foi construir um sistema em C que simulasse um player de música completo, com gerenciamento de biblioteca, playlists, fila de reprodução e histórico, sempre apoiado em estruturas de dados dinâmicas. Para chegar nesse resultado, definimos os seguintes objetivos específicos:

- Implementar uma lista duplamente encadeada para a biblioteca de músicas e para as playlists;
- Usar uma pilha encadeada como histórico de reprodução;
- Usar uma fila encadeada como fila de reprodução;
- Integrar a reprodução de áudio de verdade por meio da biblioteca miniaudio;
- Implementar o controle de teclado sem bloqueio funcionando em mais de um sistema operacional;
- Organizar o código de forma modular, com um diretório por responsabilidade.

1.2 DESCRIÇÃO DO PROBLEMA

O sistema simula um player de música completo operado pelo terminal. O usuário cadastra músicas informando título, artista e caminho do arquivo, e a duração é

descoberta automaticamente pelo engine de áudio. Essas músicas ficam guardadas em uma lista duplamente encadeada que representa a biblioteca, o que permite navegar nos dois sentidos.

Durante a reprodução, dá para avançar ou retroceder faixas, pular para a próxima música da fila, ligar o modo aleatório (shuffle) ou pausar e retomar. Toda música que toca é empilhada no histórico automaticamente. O sistema também deixa criar playlists, adicionar músicas nelas e tocar a partir delas.

Uma regra importante do sistema é que uma música não pode ser removida da biblioteca enquanto ainda estiver sendo referenciada no histórico, na fila de reprodução ou em alguma playlist. Isso garante que os ponteiros guardados nessas estruturas continuem válidos.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo apresentamos os conceitos que serviram de base para o desenvolvimento do sistema, com apoio nas referências utilizadas (BHARGAVA, 2017; BARONI, 2026).

2.1 ESTRUTURA DE DADOS: LISTA DUPLAMENTE ENCADEADA

A lista duplamente encadeada é uma estrutura linear na qual cada nodo guarda dois ponteiros, um para o nodo seguinte (`prox`) e outro para o anterior (`ant`). Por causa dessa bidirecionalidade, conseguimos percorrer a lista nas duas direções sem precisar voltar sempre ao início.

Cada nodo é alocado dinamicamente, então não existe um tamanho máximo definido em tempo de compilação. Quando já temos um ponteiro para o nodo alvo, a inserção e a remoção no meio da lista custam $O(1)$, pois basta ajustar os ponteiros dos vizinhos. No nosso sistema essa estrutura aparece tanto na biblioteca principal de músicas quanto na lista interna de cada playlist.

2.2 ESTRUTURA DE DADOS: PILHA ENCADEADA

A pilha encadeada segue a política LIFO (Last In, First Out) usando uma lista simplesmente encadeada, em que o topo é sempre o primeiro nodo. As operações de empilhar (`push`) e desempilhar (`pop`) custam $O(1)$, já que mexem apenas no topo. Como ela é encadeada, também não tem limite fixo de capacidade e cresce conforme a demanda.

No player, a pilha guarda ponteiros para os `NodoMusica` da biblioteca e funciona como histórico de reprodução. Guardar ponteiros em vez de cópias evita duplicar dados e faz com que o histórico aponte sempre para as informações originais da biblioteca.

2.3 ESTRUTURA DE DADOS: FILA ENCADEADA

A fila encadeada segue a política FIFO (First In, First Out) usando uma lista simplesmente encadeada com ponteiros para o início e para o fim. O `enqueue` insere no fim em $O(1)$ e o `dequeue` remove do início em $O(1)$. Assim como a pilha, ela não tem limite fixo de capacidade.

No player, a fila de reprodução guarda ponteiros para `NodoMusica` e define a ordem das próximas músicas. Quando a fila não está vazia, ela tem prioridade sobre a sequência da biblioteca ou da playlist.

2.4 GERENCIAMENTO DE MEMÓRIA E POLÍTICA DE REMOÇÃO

Um ponto central do projeto é a diferença entre proprietário e referenciador. A biblioteca é a dona dos `NodoMusica`, ou seja, só ela aloca e libera esses nodos. As outras estruturas (pilha, fila e playlists) guardam apenas ponteiros para eles. É por isso que uma música não pode ser removida da biblioteca enquanto houver estruturas que a referenciem. Se isso acontecesse, sobrariam ponteiros apontando para memória já liberada, o que levaria a comportamento indefinido.

2.5 A ESCOLHA DAS ESTRUTURAS

A lista duplamente encadeada foi escolhida para a biblioteca porque a navegação nos dois sentidos é essencial. Avançar e voltar na ordem das músicas são operações igualmente comuns. A pilha foi a opção natural para o histórico, já que a última música tocada é sempre a primeira que se quer consultar. E a fila representa bem a ideia de uma lista de reprodução agendada, em que a primeira música enfileirada é a primeira a tocar.

3 PLANEJAMENTO INICIAL

3.1 INTERPRETAÇÃO DO PROBLEMA

A partir do enunciado, identificamos as principais entidades do sistema: `NodoMusica` (com título, artista, caminho do arquivo e duração), `ListaDupla` (a biblioteca de músicas), `Pilha` (o histórico), `Fila` (a fila de reprodução), `NodoPlaylist` (com nome e lista interna de músicas) e `EstadoPlayer` (o estado atual da reprodução).

O fluxo básico do sistema ficou assim:

1. Cadastro de músicas pelo usuário, com a duração detectada automaticamente pelo áudio;
2. Reprodução de uma música, o que envolve carregar o arquivo no engine de áudio, empilhar no histórico e atualizar o estado;
3. Loop de reprodução ao vivo, atualizando a barra de status e lendo as teclas sem bloquear;
4. Ao terminar uma música, verificar a fila, a playlist ou a biblioteca para decidir a próxima faixa;
5. Gerenciamento de playlists, com criação, adição e remoção de músicas e reprodução a partir delas;
6. Remoção segura de músicas, checando o uso em pilha, fila e playlists antes de liberar o nodo.

3.2 ESTRUTURAS QUE SERÃO UTILIZADAS

Depois dessa análise, decidimos por:

- **Lista Duplamente Encadeada:** para a biblioteca de músicas e para a lista interna de cada playlist, por causa da navegação nos dois sentidos.
- **Pilha Encadeada:** para o histórico de reprodução, já que o acesso mais recente é o mais relevante.
- **Fila Encadeada:** para a fila de reprodução, respeitando a ordem FIFO.
- **Lista de Playlists:** também uma lista duplamente encadeada de `NodoPlaylist`, em que cada nodo tem uma lista interna de `EntradaPlaylist` (os invólucros com ponteiro para `NodoMusica`).

- **Structs em tipos.h:** para centralizar todas as definições de tipos e evitar dependência circular entre os módulos.

4 IMPLEMENTAÇÃO

Todo o desenvolvimento foi feito de forma colaborativa. Os dois integrantes discutiram cada decisão de projeto e escreveram o código juntos, usando pair programming.

4.1 INICIALIZAÇÃO DO REPOSITÓRIO DO GITHUB

Para controle de versão e colaboração, criamos um repositório privado no GitHub. A cada módulo novo fazíamos commits com mensagens descritivas, o que deixou o histórico do desenvolvimento rastreável e permitiu reverter alterações quando foi preciso.

4.2 ESTRUTURA DE ARQUIVOS E MAKEFILE

Adotamos uma organização modular, com um diretório para cada componente:

- `tipos.h`: definições centrais de todos os tipos (`NodoMusica`, `Pilha`, `Fila`, `ListaPlaylists`, `EstadoPlayer`);
- `biblioteca_func/`: lista duplamente encadeada da biblioteca;
- `pilha_func/`: pilha encadeada (histórico);
- `fila_func/`: fila encadeada (fila de reprodução);
- `playlist_func/`: lista de playlists e suas operações;
- `audio_func/`: wrapper sobre a biblioteca miniaudio;
- `input_func/`: teclado não bloqueante (Windows e Linux);
- `player_func/`: loop de reprodução ao vivo;
- `menu_func/`: todos os menus de texto;
- `main.c`: ponto de entrada.

O Makefile detecta sozinho a plataforma (Linux ou Windows via MinGW) e ajusta o executável gerado (`player` ou `player.exe`), as bibliotecas de link (`-lpthread -lm -ldl` no Linux e `-lole32 -lwinmm` no Windows) e o comando de limpeza. As flags `-Wall -Wextra -std=c99 -g` garantem rigor na compilação. Também criamos uma regra extra, `make windows`, que permite cross-compilar do Linux para Windows usando o MinGW-w64.

4.3 IMPLEMENTAÇÃO DA BIBLIOTECA (LISTA DUPLAMENTE ENCADEADA)

A biblioteca é uma `ListaDupla` com ponteiros `inicio` e `fim` e um contador `tam`. Cada `NodoMusica` é alocado dinamicamente e inserido no fim da lista. A busca por título e por artista usa correspondência parcial sem diferenciar maiúsculas de minúsculas, convertendo as strings para minúsculo em buffers temporários antes de chamar `strstr`. Já a remoção verifica, por meio de um padrão de callback, se o nodo está sendo usado em outra estrutura antes de liberar a memória.

4.4 IMPLEMENTAÇÃO DA PILHA ENCADEADA

A pilha é uma lista simplesmente encadeada com um ponteiro `topo`. Cada `NodoPilha` guarda apenas um ponteiro para `NodoMusica`, nunca uma cópia. O `push` aloca um `NodoPilha` novo e o encadeia no topo. O `pop` remove e libera esse `NodoPilha`, mas jamais o `NodoMusica` apontado por ele. A função `pilha_liberar` segue a mesma ideia: libera só os `NodoPilha` e deixa os `NodoMusica` intactos.

4.5 IMPLEMENTAÇÃO DA FILA ENCADEADA

A fila é uma lista simplesmente encadeada com ponteiros `inicio` e `fim`. Assim como a pilha, ela guarda ponteiros para `NodoMusica` sem duplicar os dados. O `enqueue` aloca um `NodoFila` e o encadeia no fim, enquanto o `dequeue` remove do início e devolve o ponteiro da música. Na lógica de avançar para a próxima música, a fila tem prioridade dentro do loop do player.

4.6 IMPLEMENTAÇÃO DAS PLAYLISTS

As playlists ficaram organizadas em uma `ListaPlaylists`, que é uma lista duplamente encadeada de `NodoPlaylist`. Cada playlist guarda por dentro uma lista duplamente encadeada de `EntradaPlaylist`, que são invólucros (wrappers) com um ponteiro para `NodoMusica` e ponteiros `prox/ant` próprios. Essa separação foi necessária porque os ponteiros `prox` e `ant` do `NodoMusica` já estão sendo usados pela cadeia da biblioteca. Se reaproveitássemos esses mesmos ponteiros nas listas internas das playlists, a biblioteca acabaria corrompida.

4.7 INTEGRAÇÃO DE ÁUDIO REAL

A reprodução foi integrada com a biblioteca miniaudio (o arquivo único `miniaudio.h`), um engine de áudio multiplataforma de código aberto escrito em C. Criamos um módulo `audio_func/` que encapsula toda a API do miniaudio e expõe só funções de alto nível: `audio_init`, `audio_load`, `audio_play`, `audio_pause`,

`audio_resume`, `audio_seek_relative` e `audio_stop`. Nenhum outro módulo inclui o `miniaudio.h` diretamente, o que isola a complexidade da biblioteca em um lugar só.

4.8 CONTROLE DE TECLADO NÃO BLOQUEANTE

O módulo `input_func/` faz a leitura de teclado sem bloqueio nos dois sistemas. No Linux, o terminal entra em modo raw com `tcsetattr`, desligando o modo canônico e o eco, e o `select` confere se há bytes disponíveis antes de chamar `read`. No Windows, usamos `_kbhit` e `_getch` da `conio.h`. A função `input_modo_normal` é registrada com `atexit`, o que garante a restauração do terminal mesmo se o programa sair de forma abrupta.

4.9 LOOP DE REPRODUÇÃO AO VIVO

O `player_loop` atualiza a barra de status do terminal a cada 200 ms, ou seja, cinco vezes por segundo, sobrescrevendo a linha atual com `\r` e chamando `fflush(stdout)`. Nesse mesmo intervalo, ele verifica se a música terminou (com `audio_is_at_end`) para avançar sozinho, e também checa se há tecla disponível para tratar os comandos do usuário. A lógica de próxima música segue a prioridade: primeiro a fila de reprodução, depois a próxima da playlist atual (ou uma aleatória, no shuffle), e por último a próxima da biblioteca (ou aleatória, no shuffle).

4.10 TESTES E VALIDAÇÕES

Fizemos testes manuais para validar:

- Inserção e remoção na lista duplamente encadeada, conferindo os ponteiros `prox` e `ant` nos extremos e no meio da lista;
- Empilhamento e desempilhamento, confirmando que só os `NodoPilha` são liberados e que os `NodoMusica` continuam intactos;
- Enfileiramento e desenfileiramento, verificando a ordem FIFO;
- Tentativa de remover músicas que estavam em uso na pilha, na fila e em playlists, confirmando as mensagens de bloqueio;
- Reprodução real de arquivos `.mp3` e `.flac`, checando a detecção de duração e o avanço e retrocesso de posição;
- Restauração do terminal depois de sair com `q` ou de uma interrupção abrupta.

4.11 DIFICULDADES E APRENDIZADOS

Durante o desenvolvimento apareceram várias dificuldades que exigiram pesquisa.

4.11.1 Gerenciamento de Propriedade de Memória

Saber quem é dono de um nodo (a biblioteca) e quem apenas o referencia (pilha, fila e playlists) foi algo que exigiu atenção o tempo todo. Liberar um `NodoMusica` enquanto ainda existiam ponteiros para ele causaria comportamento indefinido. A solução foi implementar a verificação de uso antes de qualquer remoção da biblioteca e garantir que `pilha_liberar` e `fila_liberar` nunca liberem os `NodoMusica`.

4.11.2 Dependência Circular entre Módulos

A função `biblioteca_remove` precisa saber se uma música está em alguma playlist, mas o módulo `biblioteca_func` não pode incluir `playlist_func` (que inclui o `tipos.h` e fecharia um ciclo). Resolvemos isso com um parâmetro de callback: a `biblioteca_remove` recebe um ponteiro de função `em_alguna_playlist` e um ponteiro genérico `void*` para a lista de playlists. Quem chama (o menu) passa `playlist_em_alguna` e `&playlists`, e assim os módulos ficam desacoplados.

4.11.3 Wrapper de EntradaPlaylist

Cada `NodoMusica` tem ponteiros `prox` e `ant` que pertencem à cadeia da biblioteca. Se uma playlist usasse esses mesmos ponteiros para encadear as músicas internamente, as duas listas se corromperiam. Por isso criamos o tipo `EntradaPlaylist`, que é um invólucro com seus próprios `prox` e `ant` e um ponteiro para o `NodoMusica` correspondente. Com isso, a mesma música pode aparecer em várias playlists sem nenhum conflito.

4.11.4 Integração da Biblioteca miniaudio

A `miniaudio` é uma biblioteca header-only, então a macro `MINIAUDIO_IMPLEMENTATION` precisa ser definida em exatamente um arquivo `.c` antes de incluir o header. Defini-la em mais de um arquivo geraria erro de símbolos duplicados no linker. Para evitar isso, definimos essa macro só no `audio_func/audio.c` e deixamos uma regra específica no Makefile para compilar esse arquivo sem as flags de aviso extras, que geram muitos alertas quando aplicadas a código de terceiros.

4.11.5 Teclado Não Bloqueante Multiplataforma

No Linux, a entrada do terminal é canônica por padrão, ou seja, os caracteres ficam no buffer até o usuário apertar ENTER. Para o loop ao vivo precisávamos ler tecla por tecla e sem bloquear. Implementamos o `input_modo_raw` com `tcgetattr/tcsetattr` para desligar o modo canônico e o eco, e usamos `select` com timeout zero para checar a disponibilidade sem travar. No Windows, o `_kbhit/_getch` já entrega esse comportamento de fábrica.

4.11.6 Restauração Segura do Terminal

Se o programa fosse encerrado de repente (com Ctrl+C, por exemplo) enquanto o terminal estivesse em modo raw, o usuário ficaria com o terminal inutilizável. Registramos o `input_modo_normal` com `atexit`, o que garante a restauração em qualquer caminho de saída.

4.11.7 Pausa Portável sem usleep

Para esperar 200 ms entre as iterações do loop sem usar `usleep` (que exige `_BSD_SOURCE` ou `_XOPEN_SOURCE >= 500` e nem existe no Windows), criamos uma função auxiliar `dormir_ms` baseada no `nanosleep`, que está disponível no POSIX.1b. No Windows usamos o `Sleep` da `windows.h`, escolhido pela macro `_WIN32`.

4.11.8 Tratamento de Buffer de Entrada nos Menus

O mesmo problema do trabalho anterior voltou a aparecer: o `scanf` deixa o caractere `\n` no buffer, o que atrapalha a leitura seguinte. A solução foi consumir todos os caracteres que sobraram com `getchar()` em um laço até achar `\n` ou EOF, evitando o `fflush(stdin)`, que tem comportamento indefinido fora do Windows.

4.12 DECISÕES FEITAS DURANTE O DESENVOLVIMENTO

Ao longo do projeto tomamos as seguintes decisões de arquitetura e design:

1. **Tipos centralizados em tipos.h:** ao colocar todos os `typedef` em um único arquivo incluído por todos os módulos, evitamos espalhar forward declarations e deixamos o gerenciamento de dependências mais simples.
2. **Pilha e fila guardam ponteiros, não cópias:** duplicar os `NodoMusica` em cada estrutura gastaria memória à toa e ainda exigiria sincronizar tudo sempre que um dado mudasse. Guardando ponteiros, todas as estruturas compartilham a mesma fonte de verdade, que é a biblioteca.

3. **Wrapper EntradaPlaylist para playlists:** como os ponteiros de encadeamento de um `NodoMusica` já estão em uso pela biblioteca, criar um invólucro separado foi o que permitiu a mesma música aparecer em várias playlists sem corromper nenhuma lista.
4. **Callback para verificar playlists em biblioteca_remove:** em vez de criar uma dependência direta da `biblioteca_func` sobre a `playlist_func`, passamos a função de verificação como parâmetro, mantendo o acoplamento entre os módulos no mínimo.
5. **Encapsulamento total da miniaudio:** nenhum módulo além do `audio_func` inclui o `miniaudio.h`. Isso isola a API de terceiros e facilita uma eventual troca do engine de áudio no futuro.
6. **Registro de atexit para restaurar o terminal:** garantir a restauração do terminal em qualquer caminho de saída foi uma decisão de robustez. Sem ela, uma saída por sinal ou por `exit()` direto deixaria o terminal inutilizável.
7. **Makefile multiplataforma com cross-compilação:** dar suporte a Linux e Windows no mesmo Makefile, incluindo a regra `make windows` para cross-compilar com MinGW-w64, deixou a distribuição do executável bem mais simples.

4.13 USO DE IA NO PROJETO

Usamos assistentes de inteligência artificial (ChatGPT e Claude) como ferramentas de apoio para:

- Tirar dúvidas sobre a API do `tcsetattr` e do `select` para o modo raw do terminal no Linux;
- Entender as restrições da macro `MINIAUDIO_IMPLEMENTATION` e como evitar símbolos duplicados no linker;
- Sugerir a abordagem de callback para quebrar a dependência circular entre os módulos de biblioteca e playlist;
- Ajudar na escrita do Makefile multiplataforma com detecção automática de sistema operacional.

Todo o código gerado ou sugerido por IA foi revisado, testado e integrado pela equipe, de modo que entendemos cada decisão tomada. O uso dessas ferramentas está de acordo com as diretrizes do professor, que incentiva o aprendizado crítico e a validação das informações.

5 TECNOLOGIAS UTILIZADAS

Para o desenvolvimento deste estudo dirigido, usamos as seguintes tecnologias e ferramentas:

- **Linguagem C (padrão C99)**: para implementar todo o sistema.
- **GCC (GNU Compiler Collection)**: compilador usado no Linux e no Windows (via MinGW-w64).
- **Make / mingw32-make**: automação da compilação por meio do Makefile.
- **miniaudio**: biblioteca header-only de código aberto para reprodução de áudio em C, com suporte a MP3, WAV, FLAC e outros formatos.
- **Git e GitHub**: controle de versão e hospedagem do repositório.
- **Visual Studio Code**: editor de código com extensões para C/C++.
- **Valgrind (Linux)**: ferramenta para detectar vazamentos de memória, que usamos para confirmar que só os nodos de encadeamento eram liberados, e não os `NodoMusica`.

6 DISTRIBUIÇÃO DE TAREFAS

Como o enunciado pede, apresentamos abaixo a divisão de responsabilidades entre os integrantes durante o desenvolvimento.

LUCAS TOTEROL RODRIGUES

- Implementação do módulo de fila encadeada (`fila_func/`), com todas as operações (enqueue, dequeue, peek, contem, listar) e a lógica de liberação seletiva dos nodos.
- Implementação do módulo de playlists (`playlist_func/`), com criação e remoção de playlists, adição e remoção de músicas via `EntradaPlaylist` e a navegação sequencial.
- Integração da lógica de avanço e retrocesso de músicas dentro das playlists no loop do player.

CAIO FEDERICO ESQUIVEL LOVERA ARZE

- Implementação do módulo de pilha encadeada (`pilha_func/`), com todas as operações (push, pop, peek, contem, listar) e a lógica de liberação seletiva dos nodos.
- Implementação do módulo de áudio (`audio_func/`), integrando a miniaudio e expondo a API de alto nível de reprodução, pausa, seek e detecção de fim de faixa.
- Implementação do módulo de entrada não bloqueante (`input_func/`), com suporte a Linux (`tcsetattr/select`) e Windows (`_kbhit/_getch`).

TRABALHO CONJUNTO

- Desenvolvimento da biblioteca de músicas (`biblioteca_func/`), com cadastro, remoção com verificação de uso, busca e listagem.
- Desenvolvimento do loop de reprodução ao vivo (`player_func/`), com a barra de status, o tratamento de teclas e a lógica de prioridade da próxima música.
- Desenvolvimento dos menus de texto (`menu_func/`), juntando todas as estruturas em uma interface de linha de comando coerente.
- Definição do arquivo central `tipos.h`, com todas as structs e typedefs compartilhados.

- Configuração do Makefile multiplataforma, incluindo a regra de cross-compilação para Windows.
- Realização dos testes manuais com Valgrind e validação do sistema.
- Elaboração deste relatório técnico.

7 CONSIDERAÇÕES FINAIS

Este estudo dirigido permitiu aplicar na prática os conceitos de estruturas de dados dinâmicas, lista duplamente encadeada, pilha encadeada e fila encadeada, por meio de um player de música funcional em C. A integração da reprodução de áudio de verdade, do controle de teclado sem bloqueio e das playlists deixou o projeto bem mais complexo que o trabalho anterior, o que cobrou mais rigor no gerenciamento de memória e no projeto da arquitetura.

A diferença entre proprietário e referenciador de nodos foi o conceito mais importante de todos. Entender que só a biblioteca aloca e libera os `NodoMusica`, enquanto pilha, fila e playlists apenas apontam para eles, evitou erros difíceis de depurar, como ponteiros inválidos e dupla liberação de memória. Esse padrão de propriedade clara é um princípio básico do gerenciamento manual de memória em C.

Quebrar a dependência circular entre módulos com o padrão de callback e criar invólucros (a `EntradaPlaylist`) para reutilizar estruturas em contextos diferentes foram aprendizados de design de software que vão além de simplesmente codificar uma estrutura de dados.

No fim, o projeto mostrou que escolher bem as estruturas e deixar claras as responsabilidades de cada módulo conta tanto quanto a correção do código em si. Isso serve de preparação para sistemas mais complexos, que dependem de um gerenciamento eficiente e seguro dos dados em memória.

REFERÊNCIAS

BARONI, L. A. *Estruturas de Dados - Aulas*. 2026. Slides disponibilizados na disciplina. Universidade Tuiuti do Paraná.

BHARGAVA, A. *Entendendo Algoritmos: Um Guia Ilustrado para Programadores e Outros Curiosos*. São Paulo: Novatec, 2017.